

ADVANCED DATA MANAGEMENT 2021/2022
January 10, 2022
CLOSED BOOK

Q3. (up to 10 lines) [**only for COMPUTER ENGINEERING STUDENTS**]
Discuss the CAP theorem and its impact on distributed data systems

Q2. (up to 10 lines)
Illustrate the aggregate-oriented design methodology, pointing out the goal, the input, the output and the main steps.

Q3. (up to 10 lines)

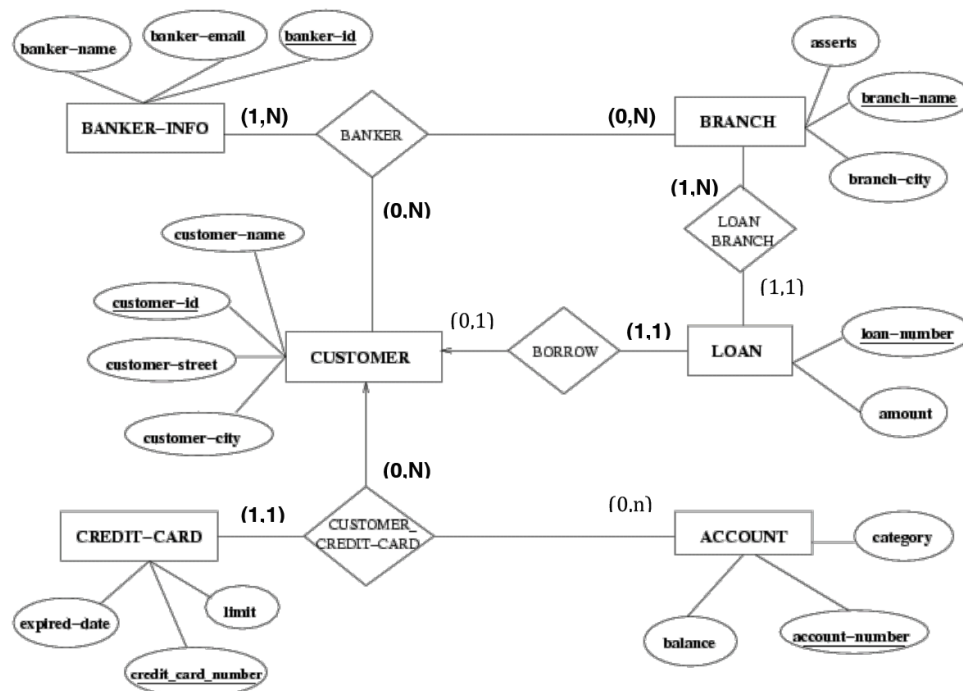
Discuss and compare how associations can be modeled in aggregate-based and graph-based NoSQL systems.

Q4. (up to 10 lines) [**only for COMPUTER SCIENCE STUDENTS**]

Describe the main features of RDF data stores.

Exercise 1 [NoSQL aggregate-oriented data stores]

Consider the following entity relationship schema:



The workload is the following:

- **Query 1:** Determine all personal customer information.
- **Query 2:** Determine the customers that borrowed a loan whose amount is greater than 5.000 euros, from a bank branch in Genova.
- **Query 3:** Determine all the loans granted by a bank branch located in Genova, Savona, Imperia, or La Spezia.

Based on the previous ER schema and workload:

1. Design, using the aggregate-oriented logical design methodology, the aggregates needed for answering the given workload queries.
2. Map the obtained aggregates into a set of Cassandra column families, by presenting the corresponding CQL CREATE TABLE commands.
3. Present (**in natural language**) an additional request over the created tables that cannot be specified in CQL and discuss an alternative approach to execute it over the Cassandra tables.

Exercise 2 [NoSQL graph-based data stores]

Consider a graph containing information about airports and flights, according to the following schema information:

Node types and related properties

```
(:Airport {airport_id, abbr, name})
(:City {name})
(:State {code, name})
(:Country {name, iso_code})
(:Carrier {code, description})
{:Flight {flight_number, year, month, day}}
(:Aircraft {tail_number})
(:Reason {code, description})
```

Relation types and related properties

```
(:City)-[:IN_COUNTRY]→(:Country)
(:State)-[:IN_COUNTRY]→(:Country)
(:City)-[:IN_STATE]→(:State)
(:Airport)-[:IN_CITY]→(:City)
(:Flight)-[:ORIGIN {taxi_time, dep_delay} | :DESTINATION {taxi_time,
arr_delay}]→(:Airport)
(:Flight)-[:CARRIER]→(:Carrier)
(:Flight)-[:CANCELLED_BY|:DELAYED_BY {duration}]→(:Reason)
(:Flight)-[:AIRCRAFT]→(:Aircraft)
```

1. Write in **natural language** and in **Cypher** a request over the graph such that:
 - it involves at least four node types;
 - it involves at least three relationship types;
 - it contains at least one aggregate function;
 - it contains at least one condition over a relationship property.Specify the type/schema of the result.
2. Write in **natural language** and in **Cypher** a request over the graph such that:
 - it involves a cumulative/accumulator operation over the nodes in a path.¹Specify the type/schema of the result.
3. Specify how you would represent in the above schema information related to the flight crew, i.e., the personnel operating on the aircraft when in flight, specifying the corresponding roles (e.g., pilot, co-pilot, flight attendant, ...)
4. Specify the information related to a flight above according to the RDF data model.

¹ **Syntax:** `reduce(accumulator = initial, variable IN list | expression)`

Exercise 3 [Semantic data stores] [only for COMPUTER SCIENCE STUDENTS]

The following is an RDF dataset about the Star Trek saga:

```
ex:JamesTKirk rdf:type ex:Person .
ex:JamesTKirk ex:hasName "Kirk" .
ex:JamesTKirk ex:hasFirstName "James" .
ex:JamesTKirk ex:hasMiddleName "Tiberius" .
ex:JamesTKirk ex:hasHomePlanet ex:Terra .
ex:JamesTKirk ex:isCaptainOf ex:Enterprise .
ex:JamesTKirk ex:isCaptainOf ex:EnterpriseClimbingClub .

ex:Spock rdf:type ex:Person .
ex:Spock ex:hasName "Spock" .
ex:Spock ex:hasHomePlanet ex:Vulcan .
ex:Spock ex:isCrewMemberOf ex:Enterprise .

ex:LeonardMcCoy rdf:type ex:Person .
ex:LeonardMcCoy ex:hasName "McCoy" .
ex:LeonardMcCoy ex:hasFirstName "Leonard" .
ex:LeonardMcCoy ex:hasHomePlanet ex:Terra .
ex:LeonardMcCoy ex:isCrewMemberOf ex:Enterprise .

ex:JeanLucPicard rdf:type ex:Person .
ex:JeanLucPicard ex:hasName "Picard" .
ex:JeanLucPicard ex:hasMiddleName "Chardonnay" .
ex:JeanLucPicard ex:isCaptainOf ex:EnterpriseD .
ex:JeanLucPicard ex:isCaptainOfSinceYear 2364 .

ex:Enterprise rdf:type ex:Starship .
ex:Enterprise ex:hasName "U.S.S. Enterprise NCC-1701" .
ex:Enterprise ex:hasCrewOf 430 .
ex:Enterprise ex:launchedInYear 2245 .

ex:Enterprise ex:hadCaptain [
  ex:captain ex:ChristopherPike ;
  ex:fromYear 2251 ;
  ex:toYear 2262 ;
] .

ex:Enterprise ex:hadCaptain [
  ex:captain ex:RobertApril ;
  ex:fromYear 2245 ;
  ex:toYear 2250;
] .

ex:EnterpriseD rdf:type ex:Starship .
ex:EnterpriseD ex:hasName "U.S.S. Enterprise NCC-1701-D" .
ex:EnterpriseD ex:launchedInYear 2363 .
ex:EnterpriseD ex:hasCrewOf 1014 .
```

1. Extend the previous RDF dataset to model all classes used in the dataset and at least three properties referenced by the input dataset with prefix `ex:`, specifying their domain and range.
2. Write a SPARQL query to retrieve, for all persons, the name and (if available) any first or middle names and write down the result of the query for the given ontology.
3. Write a SPARQL query to retrieve, for all starships, the URI and (if available) the start and end date of their term as captain of the current captain (no matter when they started) and all previous captains if the start date is 2250 or later.

Hint (Cheat sheet RDF/S)

- instance relation `rdf:type`
- subclass relation `rdfs:subClassOf`
- domain of a property `rdfs:domain`
- range of a property `rdfs:range`
- subproperty `rdfs:subPropertyOf`